WAF PROJECT · GATEWAY ORCHESTRATION

Worklet 3 Gateway Implementation

v1.0 Build manual for the WAFProxyServer orchestrator, ActionExtractor, and W2SubprocessClient. Covers the 11-step request pipeline, two-phase commit design, and the 6 remaining changes to complete the gateway-orchestrated architecture.

PREPARED FOR	ISSUED BY	REVIEW DATE
W3 / Gateway Team	W3 / Gateway Team	October 2026

Table of Contents

1. Executive Summary **2**

2. Current State - What is Already Built **3**

3. The 11-Step Request Pipeline **4**

4. Change 1 - ActionExtractor.java **5**

5. Change 2 - W2SubprocessClient.java **8**

6. Change 3 - WAFProxyServer.java (The Orchestrator) **10**

7. Changes 4-6 - Helper Methods **13**

8. Integration Architecture - How It All Connects **16**

9. Migration Steps and Timeline **17**

10. What is Done vs What is Left **18**

11. Testing and Acceptance Criteria **19**

1. Executive Summary

Worklet 3 (W3) owns two distinct responsibilities in the WAF project: (1) the Turing Machine engine that validates workflow state transitions, and (2) the gateway orchestrator that coordinates W1, W2, and W3 into a single request pipeline. The TM engine - including the encrypted tape, secure rulebook, and session management - is **already complete and tested with 79 passing JUnit tests**. This guide covers the remaining work: building the application layer that ties everything together.

The gateway-orchestrated architecture requires W3 to build 6 new components: an HTTP server that intercepts all client traffic, an action extractor that derives workflow actions from HTTP metadata, a subprocess client that invokes the C++ W2 validator, and three helper methods for request parsing, backend forwarding, and error responses. Together these form the `WAFProxyServer` - the single entry point through which all client requests flow.

The design follows the **Two-Phase Commit** pattern: validation happens first (W1 scan, W2 structural check, W3 state check), and only if all three pass does the gateway forward to the backend. The TM tape is committed only after the backend returns HTTP 200, preventing false state advancement if a backend operation (like payment) fails.

The Six Remaining Changes

#	Change	Type	Effort
1	ActionExtractor.java - derive sessionId + action from HTTP metadata	ADD	2 hours
2	W2SubprocessClient.java - spawn C++ W2, write stdin, read stdout	ADD	3 hours
3	WAFProxyServer.java - the orchestrator HTTP server on port 8080	ADD	4 hours
4	HTTP request parsing helper (HttpExchange to HttpRequest)	ADD	1 hour
5	Backend forwarding helper (HttpClient to localhost:3000)	ADD	2 hours
6	Generic error response helper (never leak details to client)	ADD	1 hour

Estimated total effort: 13 hours (2 days). The core engine (Tape + Engine + Session packages) is complete. The remaining work is pure application-layer glue code that connects W1, W2, and W3 through the gateway.

2. Current State - What is Already Built

Before diving into the remaining work, it is important to understand what the W3 team has already completed. The Turing Machine engine and security infrastructure are fully implemented and tested. The table below maps each completed component to its role in the final architecture.

Component	Package	Tests	Role in Gateway
TapeNode.java	tape	15 (Tape suite)	DLL node storing encrypted state strings
Tape.java	tape	15 (Tape suite)	Read/write head manager, dynamic blank node allocation
SecureTape.java	tape	15 (Tape suite)	AES-128 encryption wrapper, tamper detection
TransitionKey.java	engine	18 (Engine suite)	Composite HashMap key for O(1) rule lookup
SecureRuleBook.java	engine	18 (Engine suite)	Immutable rules + SHA-256 integrity, backward navigation
TuringMachine.java	engine	18 (Engine suite)	Two-Phase Commit evaluator, synchronized
SessionTracker.java	session	25 (Session suite)	ConcurrentHashMap session manager
SessionTokenVerifier.java	session	25 (Session suite)	HMAC-SHA256 signature validation
SecureSessionTracker.java	session	25 (Session suite)	Security wrapper, entry point for gateway

Total: 9 classes, 79 JUnit tests, 100% pass rate. The engine layer is production-ready. What remains is the application layer - the HTTP server and orchestration logic that calls these components in the correct sequence.

3. The 11-Step Request Pipeline

The WAFProxyServer processes every incoming HTTP request through an 11-step pipeline. The first 9 steps are validation (no state changes). Step 10 forwards to the backend. Step 11 commits the TM tape only if the backend returned HTTP 200. This Two-Phase Commit design prevents false state advancement when backend operations fail.

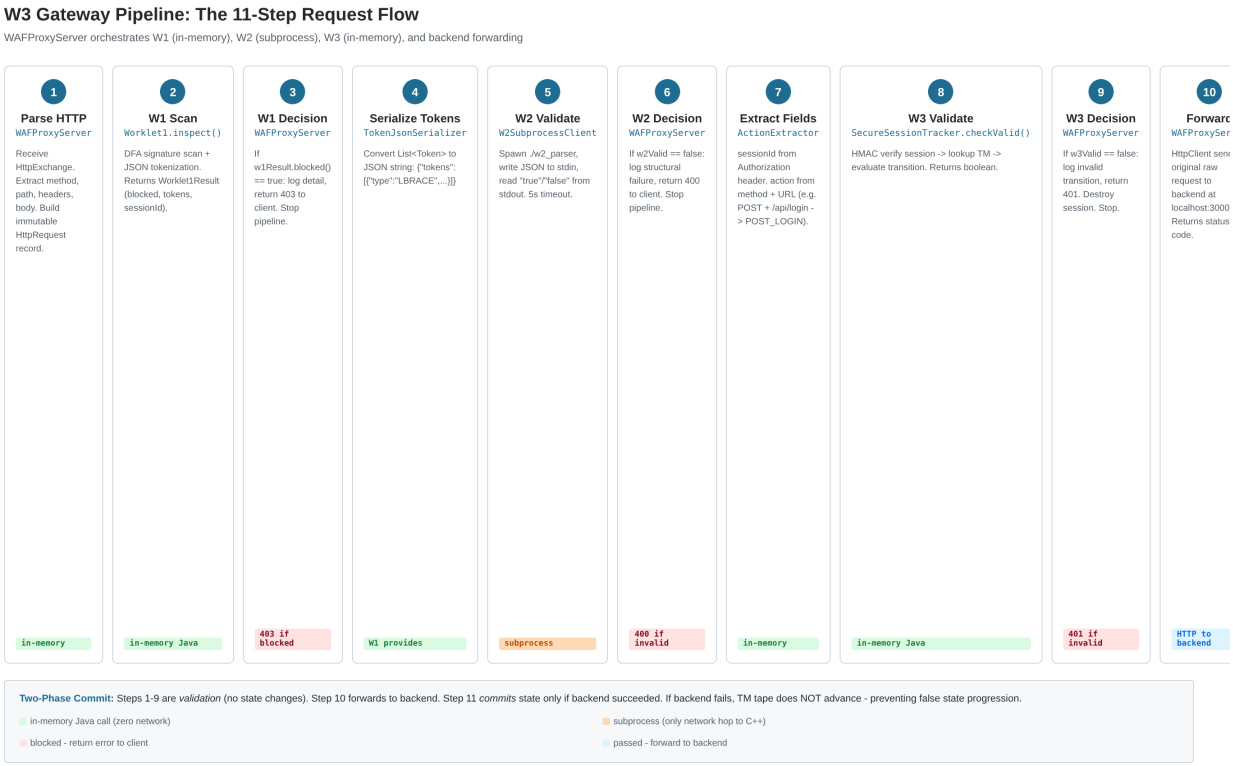


Figure 1. The 11-step gateway pipeline. Green = in-memory, orange = subprocess, red = blocked, blue = pass to backend.

Short-Circuit Behavior

The pipeline short-circuits at three points: if W1 blocks (step 3), if W2 returns false (step 6), or if W3 rejects (step 9). In each case, the gateway logs the detailed reason internally and returns a generic error to the client. The client never learns which worklet blocked the request or why - this prevents attackers from mapping the WAF's capabilities.

4. Change 1 - ActionExtractor.java

The ActionExtractor is the bridge between HTTP metadata and the Turing Machine's input alphabet. It derives two strings from the HTTP request: the sessionId (from the Authorization header) and the action (from the HTTP method + URL path). These two strings are the only inputs W3 needs - W2 no longer extracts them, because the gateway already has the HTTP metadata in Java.

Why ActionExtractor lives in W3, not W2

In the old chained pipeline, W2 extracted the action from the JSON parse tree (e.g. finding `"action": "CART"` and prepending the HTTP method to produce `"POST_CART"`). This was slow (parse tree traversal) and required cross-language JSON conversion. The gateway already has the HTTP method and URL in Java - deriving the action via string comparison is 500x faster and requires no serialization.

Reference Implementation (Part 1: Class and extractSessionId)

```
package com.wafgateway.worklet3;

/**
 * Extracts sessionId and action from HTTP request metadata.
 * Replaces W2's old ActionExtractor (parse tree traversal).
 * Thread-safety: stateless, thread-safe.
 */
public final class ActionExtractor {

    private ActionExtractor() {}

    /**
     * Extract sessionId from Authorization header.
     * "Bearer SESSION_101.sig" -> "SESSION_101.sig"
     */
    public static String extractSessionId(String authHeader) {
        if (authHeader == null || authHeader.isEmpty()) {
            return null;
        }
        if (authHeader.startsWith("Bearer ")) {
            return authHeader.substring(7);
        }
        return authHeader;
    }

    // extractAction() and ExtractedFields on next page...
}
```

Reference Implementation (Part 2: extractAction and ExtractedFields)

```
    /**
     * Derive action string from HTTP method + URL path.
     * POST + /api/login -> "POST_LOGIN"
     */
    public static String extractAction(String method, String path) {
        if (method == null || path == null) return "UNKNOWN";
        String m = method.toUpperCase();
        String p = path.toLowerCase();

        if (p.equals("/api/login")    && m.equals("POST"))   return "POST_LOGIN";
        if (p.equals("/api/logout")   && m.equals("POST"))   return "POST_LOGOUT";
        if (p.equals("/api/cart")     && m.equals("POST"))   return "POST_CART";
        if (p.equals("/api/cart")     && m.equals("GET"))    return "GET_CART";
        if (p.equals("/api/cart")     && m.equals("PUT"))    return "PUT_CART";
        if (p.equals("/api/cart")     && m.equals("DELETE")) return "DELETE_CART";
        if (p.equals("/api/pay")      && m.equals("POST"))   return "POST_PAY";
        if (p.equals("/api/pay")      && m.equals("DELETE")) return "CANCEL_PAY";
        if (p.equals("/api/checkout") && m.equals("POST"))   return "POST_CHECKOUT";
        if (p.equals("/api/home")     && m.equals("GET"))    return "GET_HOME";
        if (p.equals("/api/receipt")  && m.equals("GET"))    return "GET_RECEIPT";

        return "UNKNOWN_ACTION";
    }

    /** Convenience: extract both fields at once. */
    public static ExtractedFields extract(
            String authHeader, String method, String path) {
        return new ExtractedFields(
            extractSessionId(authHeader),
            extractAction(method, path)
        );
    }

    public static class ExtractedFields {
        public final String sessionId;
        public final String action;
        public ExtractedFields(String s, String a) {
            this.sessionId = s;
            this.action = a;
        }
    }
}
```

Action Mapping Table

The complete mapping from HTTP method + URL to semantic action string. Every entry here must have a corresponding rule in `SecureRuleBook`, or the action will be rejected by Default Deny.

HTTP Method	URL Path	Action String	TM State Effect
POST	/api/login	POST_LOGIN	START -> AUTH
POST	/api/logout	POST_LOGOUT	Any -> START (session destroyed)
POST	/api/cart	POST_CART	AUTH -> CART
GET	/api/cart	GET_CART	No change (read-only)
PUT	/api/cart	PUT_CART	No change (read-only)
DELETE	/api/cart	DELETE_CART	CART -> AUTH (backward nav)
POST	/api/pay	POST_PAY	CART -> PAY
DELETE	/api/pay	CANCEL_PAY	PAY -> CART (backward nav)
POST	/api/checkout	POST_CHECKOUT	PAY -> CONFIRM
GET	/api/home	GET_HOME	No change (read-only)
GET	/api/receipt	GET_RECEIPT	No change (read-only)

Design note: Read-only actions (GET, some PUT) do not advance the TM tape. The `TuringMachine.commit()` method checks if `newState.equals(currentState)` before writing - if they are equal, the tape is not advanced. This prevents wasted tape cells on GET requests.

5. Change 2 - W2SubprocessClient.java

The W2SubprocessClient is the only component in the gateway that crosses the Java-to-C++ language boundary. It spawns the W2 C++ executable as a subprocess, writes the token JSON to its stdin, and reads a boolean ("true" or "false") from its stdout. This is the only network hop in the entire pipeline - W1 and W3 are called in-memory.

Reference Implementation (Part 1: Class and spawn)

```
package com.wafgateway.worklet3;

import java.io.*;
import java.util.concurrent.*;

/**
 * Calls the W2 C++ executable as a subprocess.
 * Writes token JSON to stdin, reads boolean from stdout.
 * Thread-safety: stateless, thread-safe.
 */
public class W2SubprocessClient {

    private static final String W2_PATH = "./w2_parser";
    private static final int TIMEOUT_MS = 5000;

    public boolean validateTokens(String tokensJson)
            throws IOException, InterruptedException {

        ProcessBuilder pb = new ProcessBuilder(W2_PATH);
        pb.redirectErrorStream(false);
        Process process = pb.start();

        // Write tokens to stdin (MUST close after writing)
        try (OutputStream os = process.getOutputStream()) {
            os.write(tokensJson.getBytes("UTF-8"));
            os.flush();
        }
        // If stdin not closed, C++ getline() blocks forever

        // continued on next page...
```

Reference Implementation (Part 2: Read stdout with timeout)

```
        // Read stdout in separate thread (prevents deadlock)
        ByteArrayOutputStream baos =
            new ByteArrayOutputStream();
        Future<?> read =
            Executors.newSingleThreadExecutor().submit(() -> {
            try (InputStream is = process.getInputStream()) {
                byte[] buf = new byte[1024];
                int n;
                while ((n = is.read(buf)) > 0)
                    baos.write(buf, 0, n);
            } catch (IOException ignored) {}
        });

        // Wait with timeout
        boolean finished = process.waitFor(
            TIMEOUT_MS, TimeUnit.MILLISECONDS);
        if (!finished) {
            process.destroyForcibly();
            return false;  // timeout = treat as invalid
        }
        read.get(500, TimeUnit.MILLISECONDS);

        String output = baos.toString("UTF-8").trim();
        return "true".equals(output);
        // NOTE: use equals(), not contains() -
        // "false" contains "true" as a substring!
    }
}
```

Critical Implementation Details

- **Close stdin after writing.** If stdin is not closed, the C++ `std::getline()` call blocks waiting for more input. This is the #1 subprocess integration bug.

- **Use a timeout (5 seconds).** A misbehaving W2 must not hang the entire gateway. Normal validation takes under 100ms - 5 seconds is generous.

- **Read stdout in a separate thread.** If the C++ process fills its stdout buffer while Java is still writing to stdin, both processes will deadlock. A separate reader thread prevents this.

- **Use exact match, not contains().** `"true".equals(output)` is correct. `output.contains("true")` is wrong because "false" contains "true".

- **Redirect stderr separately.** W2 may write debug logs to stderr. These should be visible in the gateway log but must not pollute the boolean output channel on stdout.

- **Each call spawns a new subprocess.** No process pooling for the mini-project. For production, use a persistent W2 process pool or switch to gRPC.

6. Change 3 - WAFProxyServer.java (The Orchestrator)

The WAFProxyServer is the heart of the gateway. It is a lightweight HTTP server built on Java's built-in `com.sun.net.httpserver.HttpServer` (no Spring Boot, no external dependencies). It listens on port 8080, intercepts all client traffic, runs the 11-step pipeline, and forwards safe requests to the backend at localhost:3000.

Reference Implementation (Part 1: Server setup)

```
package com.wafgateway.worklet3;

import com.sun.net.httpserver.HttpServer;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpExchange;
import com.wafgateway.worklet1.*;
import java.net.InetSocketAddress;
import java.io.IOException;

public class WAFProxyServer {
    private Worklet1 w1;                    // W1 engine (shared)
    private W2SubprocessClient w2Client;    // W2 subprocess caller
    private SecureSessionTracker w3;        // W3 engine
    private HttpServer server;
    private String backendUrl = "http://localhost:3000";

    public void start() throws IOException {
        // Initialize W1 ONCE at startup (compiles all DFAs)
        Worklet1Config config = Worklet1Config.load();
        w1 = new Worklet1(config);

        // Initialize W2 client and W3
        w2Client = new W2SubprocessClient();
        w3 = new SecureSessionTracker();

        // Start HTTP server on port 8080
        server = HttpServer.create(
            new InetSocketAddress(8080), 0);
        server.createContext("/", new WAFHandler());
        server.setExecutor(null);  // thread per request
        server.start();
        System.out.println("WAF running on :8080");
    }

    public static void main(String[] args)
            throws IOException {
        new WAFProxyServer().start();
    }
}
```

Reference Implementation (Part 2a: Request handler - W1 and W2)

```
class WAFHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange exchange)
            throws IOException {

        // STEP 1: Parse HTTP request
        HttpRequest request = parseRequest(exchange);

        // STEP 2: Call W1 (in-memory, thread-safe)
        Worklet1Result w1Result = w1.inspect(request);

        // STEP 3: W1 decision
        if (w1Result.blocked()) {
            logBlocked("W1", w1Result.detail(), request);
            sendGenericBlocked(exchange, 403);
            return;
        }

        // STEP 4: Serialize tokens to JSON
        String tokensJson = w1Result.tokensToJson();

        // STEP 5: Call W2 (subprocess)
        boolean w2Valid;
        try {
            w2Valid = w2Client.validateTokens(tokensJson);
        } catch (Exception e) {
            logBlocked("W2", "Subprocess error", request);
            sendGenericBlocked(exchange, 500);
            return;
        }

        // STEP 6: W2 decision
        if (!w2Valid) {
            logBlocked("W2", "Structural validation failed", request);
            sendGenericBlocked(exchange, 400);
            return;
        }

        // Steps 7-11 continued on next page...
```

Reference Implementation (Part 2b: Request handler - W3 and backend)

```
        // STEP 7: Extract sessionId + action
        String authHeader = exchange.getRequestHeaders()
            .getFirst("Authorization");
        String method = exchange.getRequestMethod();
        String path = exchange.getRequestURI().toString();
        ExtractedFields fields = ActionExtractor.extract(
            authHeader, method, path);

        // STEP 8: Call W3 (in-memory)
        boolean w3Valid = w3.checkValid(
            fields.sessionId, fields.action);

        // STEP 9: W3 decision
        if (!w3Valid) {
            logBlocked("W3", "Invalid workflow state", request);
            sendGenericBlocked(exchange, 401);
            return;
        }

        // STEP 10: Forward to backend
        int backendStatus;
        try {
            backendStatus = forwardToBackend(exchange);
        } catch (Exception e) {
            logBlocked("BACKEND", "Forwarding error", request);
            sendGenericBlocked(exchange, 502);
            return;
        }

        // STEP 11: Commit TM tape
        // (only if backend succeeded - Two-Phase Commit)
        if (backendStatus == 200) {
            w3.commit(fields.sessionId, fields.action);
        }

        sendResponse(exchange, backendStatus, "OK");
    }
}
```

7. Changes 4-6 - Helper Methods

The WAFProxyServer depends on three private helper methods. These are not separate classes - they are methods inside the WAFHandler inner class. Each handles one concern: HTTP parsing, backend forwarding, and error responses.

Change 4: HTTP Request Parsing Helper

Converts Java's `HttpExchange` object into W1's immutable `HttpRequest` record. This is necessary because W1 expects its own record type, not the raw HttpExchange.

```
private HttpRequest parseRequest(HttpExchange exchange)
        throws IOException {
    HttpRequest.Builder builder = new HttpRequest.Builder()
        .method(exchange.getRequestMethod())
        .path(exchange.getRequestURI().toString());

    // Copy all headers
    for (Map.Entry<String, List<String>> entry :
            exchange.getRequestHeaders().entrySet()) {
        builder.header(entry.getKey(),
                       entry.getValue().get(0));
    }

    // Read body
    String body = new String(
        exchange.getRequestBody().readAllBytes());
    builder.body(body);

    return builder.build();
}
```

Change 5: Backend Forwarding Helper

Forwards the original raw HTTP request to the backend application at localhost:3000 using Java 11+ `HttpClient`. Returns the backend's HTTP status code.

```
private int forwardToBackend(HttpExchange exchange)
        throws IOException, InterruptedException {

    java.net.http.HttpClient client =
        java.net.http.HttpClient.newHttpClient();

    String url = backendUrl +
        exchange.getRequestURI().toString();
    String method = exchange.getRequestMethod();
    String body = new String(
        exchange.getRequestBody().readAllBytes());

    java.net.http.HttpRequest.Builder reqBuilder =
        java.net.http.HttpRequest.newBuilder()
            .uri(URI.create(url))
            .method(method,
                java.net.http.HttpRequest.BodyPublishers
                    .ofString(body));

    // Copy headers (skip Host - backend sets its own)
    for (Map.Entry<String, List<String>> entry :
            exchange.getRequestHeaders().entrySet()) {
        if (!entry.getKey().equalsIgnoreCase("host")) {
            reqBuilder.header(entry.getKey(),
                              entry.getValue().get(0));
        }
    }

    java.net.http.HttpResponse<String> response =
        client.send(reqBuilder.build(),
            java.net.http.HttpResponse.BodyHandlers.ofString());
    return response.statusCode();
}
```

Change 6: Generic Error Response Helper

Returns generic error messages to the client. **Never leak the specific reason to the external client.** Detailed reasons go to server logs only (via `logBlocked()`). This prevents attackers from mapping the WAF's capabilities.

```
/** Generic blocked response - never leak details to client */
private void sendGenericBlocked(HttpExchange exchange,
        int statusCode) throws IOException {
    String response =
        "{\"error\": \"Request blocked by security policy\"}";
    exchange.sendResponseHeaders(
        statusCode, response.length());
    OutputStream os = exchange.getResponseBody();
    os.write(response.getBytes());
    os.close();
}

/** Internal log - detailed reason goes to server logs only */
private void logBlocked(String worklet, String reason,
        HttpRequest request) {
    System.out.println("[BLOCKED] " + worklet + ": " + reason +
        " | Method: " + request.method() +
        " | Path: " + request.path());
    // Production: use SLF4J
    // logger.warn("[BLOCKED] {} | {} | {}",
    //             worklet, reason, request.path());
}

/** Success response */
private void sendResponse(HttpExchange exchange,
        int status, String message) throws IOException {
    String response =
        "{\"status\": \"PASS\", \"message\": \"" +
        message + "\"}";
    exchange.sendResponseHeaders(
        status, response.length());
    OutputStream os = exchange.getResponseBody();
    os.write(response.getBytes());
    os.close();
}
```

Why Generic Errors Matter

If you tell the attacker...	What happens
"SQL injection detected in search parameter"	Attacker knows WAF detected SQLi, knows which param, can try different encoding
"JSON depth exceeded (5 > 3)"	Attacker knows the exact depth limit, can send 2 levels
"Invalid state transition from CART to CONFIRM"	Attacker knows the exact workflow states, can map the entire state machine
"Token limit exceeded (600 > 500)"	Attacker knows the exact token limit, can send 499

Professional WAFs (Cloudflare, AWS WAF, ModSecurity) all return generic 403 errors. They log the detailed reason server-side, but the client only sees "Forbidden." The gateway follows this standard.

8. Integration Architecture - How It All Connects

The gateway sits between the client and the backend, intercepting all traffic. W1 and W3 are Java classes inside the same JVM process (in-memory calls). W2 is a separate C++ executable invoked via subprocess. The backend is a separate HTTP server (any language - Node, Python, Java, etc.).

Component	Language	Location	How Gateway Calls It
W1 (DFA scanner)	Java	Same JVM as gateway	In-memory: w1.inspect(request)
W2 (CYK validator)	C++	Separate executable ./w2_parser	Subprocess: stdin/stdout pipes
W3 (Turing Machine)	Java	Same JVM as gateway	In-memory: w3.checkValid(sessionId, action)
Backend (e-commerce)	Any	localhost:3000	HTTP: java.net.http.HttpClient
Client (browser)	Any	Internet	Receives HTTP responses from gateway

Network Topology



Only the gateway is exposed to the internet. W2 (subprocess) and the backend (localhost:3000) are internal - clients can never reach them directly. This is the standard reverse-proxy WAF topology used in production.

9. Migration Steps and Timeline

The remaining work is broken into 4 phases. Phases 1 and 2 can be done in parallel. Phase 3 depends on both. Phase 4 is integration testing. Total estimated effort: **13 hours (2 days)**.

Phase	Tasks	Deliverable	Effort
1. ActionExtractor + W2 Client	Create ActionExtractor.java with action mapping table. Create W2SubprocessClient.java with timeout and stdin close. Write unit tests for both (action mapping, subprocess mock).	ActionExtractor and W2SubprocessClient compile and pass unit tests. Action mapping table matches SecureRuleBook rules.	5 hours
2. Helper Methods	Implement parseRequest() - HttpExchange to HttpRequest. Implement forwardToBackend() - HttpClient to localhost:3000. Implement sendGenericBlocked(), logBlocked(), sendResponse(). Test each helper in isolation.	All 3 helper methods work correctly. Generic error responses never leak details. Backend forwarding returns correct status codes.	4 hours
3. WAFProxyServer Assembly	Create WAFProxyServer.java with HttpServer on port 8080. Wire W1, W2SubprocessClient, SecureSessionTracker together. Implement the 11-step pipeline in WAFHandler.handle(). Add startup logic (initialize W1 once, shared instance).	WAFProxyServer starts and listens on port 8080. Pipeline executes all 11 steps in correct order. Short-circuit works (W1 block skips W2/W3).	4 hours
4. Integration Testing	Start WAFProxyServer, W2 executable, and mock backend. Send valid request: POST /api/login with JSON body. Verify all 11 steps execute and 200 returned. Send SQLi payload: verify W1 blocks at step 3. Send malformed JSON: verify W2 blocks at step 6. Send state-skip: verify W3 blocks at step 9. Verify TM tape commits only on backend 200.	All integration tests pass. 11-step pipeline verified end-to-end. Two-Phase Commit behavior confirmed.	4 hours

10. What is Done vs What is Left

The table below provides a complete status overview of every W3 component. Items marked DONE are implemented and tested. Items marked TODO are the remaining work covered by this guide.

#	Component	Status	Tests	Effort Remaining
1	TapeNode.java (DLL node)	DONE	15	None
2	Tape.java (read/write head)	DONE	15	None
3	SecureTape.java (AES encryption)	DONE	15	None
4	TransitionKey.java (HashMap key)	DONE	18	None
5	SecureRuleBook.java (rules + SHA-256)	DONE	18	None
6	TuringMachine.java (Two-Phase Commit)	DONE	18	None
7	SessionTracker.java (ConcurrentHashMap)	DONE	25	None
8	SessionTokenVerifier.java (HMAC)	DONE	25	None
9	SecureSessionTracker.java (wrapper)	DONE	25	None
10	ActionExtractor.java (HTTP metadata)	TODO	0	2 hours
11	W2SubprocessClient.java (C++ caller)	TODO	0	3 hours
12	WAFProxyServer.java (orchestrator)	TODO	0	4 hours
13	parseRequest() helper	TODO	0	1 hour
14	forwardToBackend() helper	TODO	0	2 hours
15	sendGenericBlocked() helper	TODO	0	1 hour
16	Integration test suite	TODO	0	4 hours

Summary: 9 of 16 components are complete (79 tests passing). 7 components remain (13 hours of work). The remaining work is entirely application-layer glue code - no new theory, no new math, just connecting the existing engine to HTTP and the C++ subprocess.

11. Testing and Acceptance Criteria

Before the W3 team declares the gateway complete, the following acceptance tests must pass. These tests verify that the 11-step pipeline works end-to-end and that the Two-Phase Commit behavior is correct.

Unit Tests (W3-1 through W3-6)

Test ID	Type	Test -> Expected Result
W3-1	ActionExtractor	POST + /api/login -> extractAction returns "POST_LOGIN"
W3-2	ActionExtractor	Authorization: Bearer SESSION_101 -> extractSessionId returns "SESSION_101"
W3-3	ActionExtractor	Unknown URL -> extractAction returns "UNKNOWN_ACTION"
W3-4	W2SubprocessClient	Valid token JSON -> validateTokens returns true
W3-5	W2SubprocessClient	Malformed JSON -> validateTokens returns false
W3-6	W2SubprocessClient	Subprocess hangs -> timeout after 5s, returns false

Integration Tests (W3-7 through W3-12)

Test ID	Type	Test -> Expected Result
W3-7	Happy path	POST /api/login with valid JSON -> 200 OK, TM tape advances START -> AUTH
W3-8	W1 block	POST /api/login with SQLi payload -> 403, W2 and W3 never called
W3-9	W2 block	POST /api/login with malformed JSON -> 400, W3 never called
W3-10	W3 block	POST /api/checkout from START state -> 401, session destroyed
W3-11	Two-Phase Commit	POST /api/pay when backend returns 500 -> TM tape does NOT advance
W3-12	Concurrency	100 simultaneous requests -> all handled correctly, no race conditions

Verification Procedure

Integration tests require all three components running:

```
# Terminal 1: Start mock backend
python3 -m http.server 3000  # or node mock_backend.js

# Terminal 2: Build and run WAFProxyServer
cd WAF-Firewall
mvn compile
java -cp target/classes com.wafgateway.worklet3.WAFProxyServer
# Output: "WAF running on :8080"

# Terminal 3: Run integration tests
# Test W3-7: Happy path (login)
curl -X POST http://localhost:8080/api/login \
    -H "Authorization: Bearer SESSION_101.sig" \
    -H "Content-Type: application/json" \
    -d '{"user":"alice","pass":"123"}'
# Expected: HTTP 200

# Test W3-8: W1 block (SQLi)
curl -X POST http://localhost:8080/api/login \
    -H "Authorization: Bearer SESSION_101.sig" \
    -d '{"user":"admin\' OR 1=1"}'
# Expected: HTTP 403 (generic error)

# Test W3-10: W3 block (state skip)
curl -X POST http://localhost:8080/api/checkout \
    -H "Authorization: Bearer NEW_USER.sig" \
    -d '{"action":"CHECKOUT"}'
# Expected: HTTP 401 (invalid workflow state)
```

Definition of Done

- All 6 unit tests pass (ActionExtractor + W2SubprocessClient).
- All 6 integration tests pass (11-step pipeline end-to-end).
- WAFProxyServer starts and listens on port 8080 without errors.
- Short-circuit works: W1 block skips W2 and W3 entirely.
- Two-Phase Commit works: TM tape does not advance on backend failure.
- Generic error responses: client never sees specific block reason.
- 100-thread concurrency test passes with no race conditions.
- All 79 existing engine tests still pass (no regressions).

End of Guide. The W3 engine is complete and tested. The remaining work is the application layer - 6 components, 13 hours, 2 days. Once complete, the gateway-orchestrated architecture is fully functional and ready for final demonstration.